

Spring Boot

Cuando trabajamos con Spring, constantemente escuchamos el término "decorar una clase". Este término generalmente se refiere a la aplicación de anotaciones de Spring en una clase para configurar su comportamiento y funcionalidad dentro de Spring framework.

Al decorar una clase con anotaciones de Spring, estas indicando a Spring cómo debe manejar y configurar esa clase dentro de su contenedor de inversión de control (IoC), lo que permite la administración de componentes, la inyección de dependencias, la configuración de propiedades, entre otras funcionalidades.

Anotaciones

En Spring Boot, una anotación se utiliza para configurar y definir el comportamiento de los componentes de una aplicación. Simplificar el desarrollo de los componentes de una aplicación.

Estas anotaciones se utilizan para simplificar el desarrollo de aplicaciones empresariales con Java.

Definiciones de anotaciones:

@SpringBootApplication: Esta es la anotación principal que se utiliza para marcar la clase principal de una aplicación Spring Boot.

@RestController: La anotación `@RestController` se utiliza para marcar una clase Java como un controlador REST, en una aplicación Spring Boot. Esta anotación combina las funcionalidades de `@Controller` y `@ResponseBody`, lo que significa que es capaz de manejar solicitudes HTTP y devolver datos directamente en formato JSON o XML.

Se utiliza para crear controladores en aplicaciones Spring Boot que generan respuestas HTTP para APIs RESTful. Cuando una clase está anotada con `@RestController`, cada método de esa clase se mapea automáticamente a una ruta URL y responde a las solicitudes HTTP entrantes en función de las anotaciones `@GetMapping`, `@PostMapping`, `@PutMapping`, `@DeleteMapping`, etc. En resumen es una anotación clave en Spring Boot que simplifica el desarrollo de APIs RESTful al combinar la anotación `@Controller` y `@ResponseBody`, permitiendo a los desarrolladores crear controladores que generan respuestas HTTP en formato JSON o XML de manera fácil y rápida.

@GetMapping: Se utiliza para mapear las solicitudes HTTP GET a métodos de controladora específicos en una aplicación Spring Boot.

Esta anotación se utiliza comúnmente en combinación con la anotación `@RestController` o `@Controller` para crear controladores RESTful en una aplicación Spring.

Cuando se utiliza `@GetMapping`, Spring Boot asigna la URL especificada en la anotación a un método en el controlador y maneja automáticamente las solicitudes HTTP GET que llegan a esas URL, invocando el método correspondiente y devolviendo el resultado como respuesta. En resumen se utiliza en solicitudes HTTP GET a métodos de controlador específicos, lo que facilita crear APIs RESTful y manejar las solicitudes GET.

@PathVariable: En Spring boot, se utiliza para mapear partes de la URL de una solicitud web a parámetros de un controlador de Spring.

Esto es útil cuando tiene valores dinámicos en la URL que deseas extraer y utilizar en método del controlador. En resumen, la anotación `@PathVariable` indica que un parámetro Java se transformará en un parámetro web que estará contenido en la URL de un endpoint.

@PostMapping: mapea las solicitudes HTTP POST a métodos específicos.

Es uno de los métodos HTTP utilizados para enviar datos a un servidor y crear un nuevo recurso. Cuando se realiza una solicitud POST, se está solicitando al servidor que acepte los datos incluidos en el cuerpo de la solicitud y los procese de acuerdo con la lógica de la aplicación.

@PutMapping: mapea las solicitudes HTTP PUT a métodos específicos en controladores Spring. Cuando un cliente envía una solicitud HTTP del tipo PUT a una URL específica en la aplicación Spring Boot, está indicando que desea actualizar o reemplazar por completo un recurso en el servidor con los datos proporcionados en cuerpo de la solicitud.

Con lo cual, la anotación `@PutMapping` dirige la solicitud al método correspondiente en el controlador y de esta manera efectuar los cambios correspondientes.

@DeleteMapping: se utiliza para mapear métodos de controlador específicos que generalmente se usan para eliminar un recurso existente en el servidor.

Se encarga de manejar las solicitudes HTTP del tipo DELETE y enrutar la solicitud al método que hayamos especificado mediante la URL asignada.

@RequestBody: se utiliza para anotar argumentos del método del controlador de solicitudes.

Esta anotación indica que un parámetro del método debe estar vinculado al valor del cuerpo de la solicitud HTTP. Es decir se encarga que un objeto se construirá a partir de la información enviada en el body del request para la petición de este servicio.

@PathMapping: es parte de la familia de anotaciones para mapear solicitudes HTTP y se utiliza para mapear solicitudes HTTP del tipo PATH a métodos de controlador que se encargaban de realizar la modificación parcial de recursos en nuestro sistema.

Una solicitud HTTP del tipo PATH se utiliza para aplicar modificaciones parciales en recursos existentes sin necesidad de enviar la representación completa del recurso. A diferencia de PUT, que reemplaza por completo el recurso, PATCH permite actualizar solo algunos campos del recurso existente.

@RequestMapping: Se utiliza para mapear las solicitudes HTTP a los métodos de manejo específicos del controlador.

Cuando se usa a nivel de clase, proporciona una ruta base común para todos los métodos del controlador, lo que unifica el prefijo establecido en la ruta para todos los endpoints.

Mientras que usar @RequestMapping a nivel de método, permite especificar, la ruta, el método HTTP y otros detalles directamente en cada método del controlador. Lo cual es una alternativa a usar anotaciones como

@GetMapping, @PostMapping, etc. Ya que es más versátil y puede ser más expresivo cuando necesitas especificar más detalles.

@Service: al decorar una clase con @Service, indicamos al contenedor de Spring que esa clase debe ser tratada como un bean de servicio.

Esto significa que Spring gestiona automáticamente la creación de instancias que Spring gestiona automáticamente la creación de instancias de esa clase y podrá inyectar en otras partes de la aplicación cuando sea necesario.

@Autowired: es una de las más fundamentales en Spring Boot y juega un papel crucial en la implementación de la inyección de dependencias.

@Autowired le indica a Spring que debe inyectar automáticamente una dependencia en el lugar donde se declara.

Esta inyección puede hacerse en un constructor, un método setter, o directamente en un campo (atributo) de la clase.

¿Cómo funciona? le dice al contenedor de Spring que debe satisfacer la dependencias automáticamente al proporcionar una instancia del bean requerido.

Cuando Spring encuentra la anotación @Autowired, busca en su contenedor IoC una instancia del bean que coincida con el tipo del atributo, parámetro del constructor o método y lo inyecta.

Microservicios

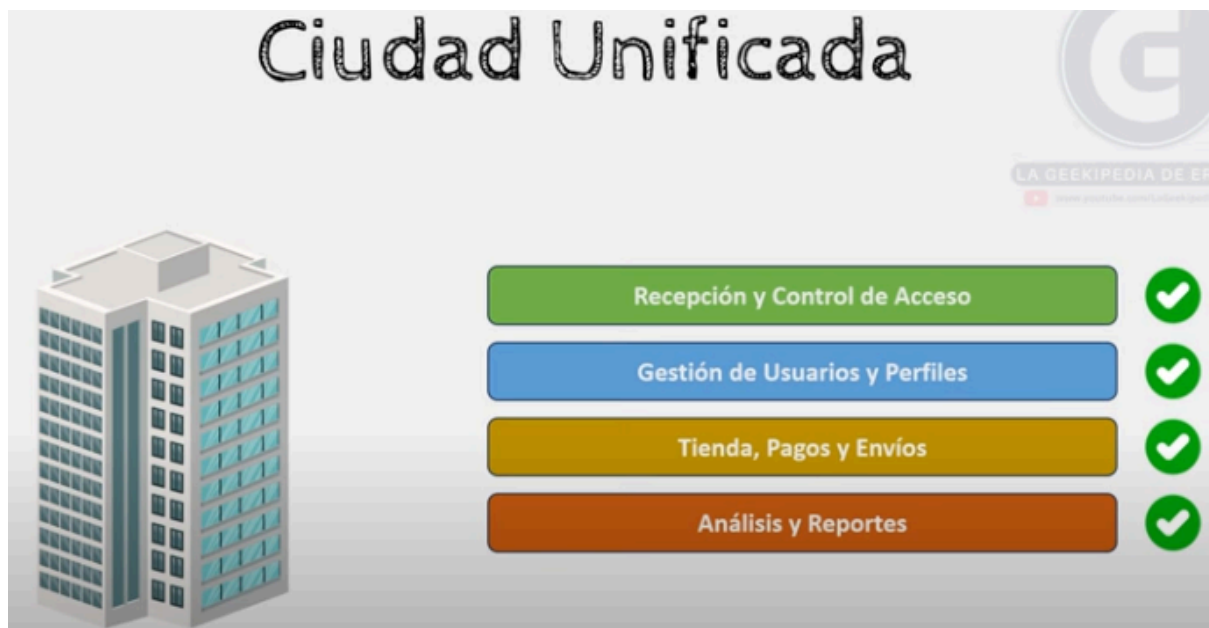
Los microservicios son una arquitectura de software en la que una aplicación monolítica se descompone en pequeños servicios independientes.

Cada una de estos servicios se centra en realizar una tarea específica, y pueden ser desarrollados, implementados y escalados de manera independiente.

La idea detrás de los microservicios es dividir una aplicación monolítica en componentes más pequeños y manejables.

Cada microservicio puede ser desarrollado utilizando diferentes tecnologías y lenguajes de programación según lo que mejor se adapte a su función específica.

Ejemplo de microservicio:

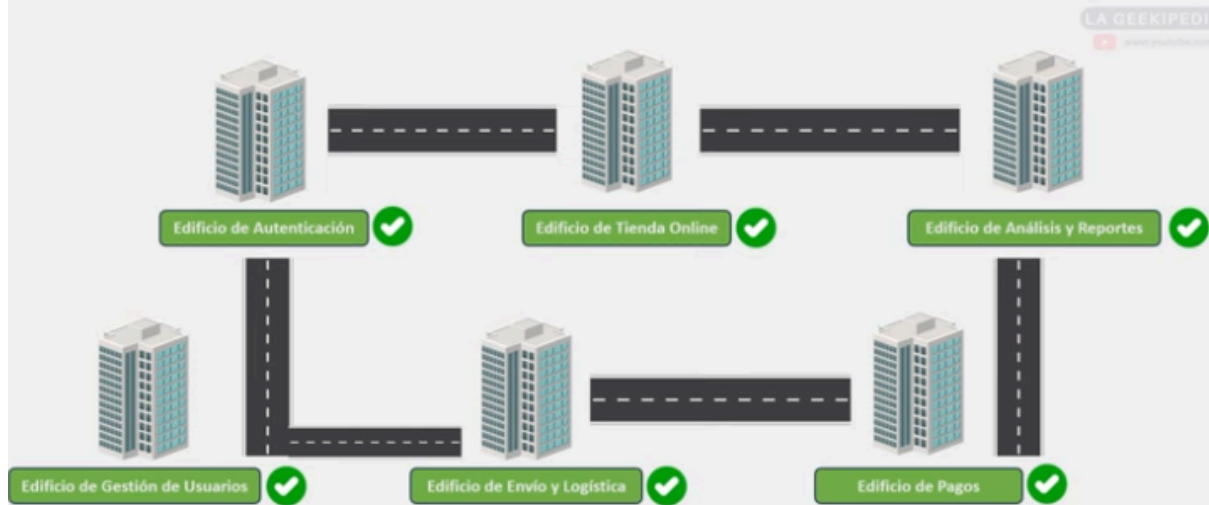


Este edificio tiene 4 departamentos en donde cada departamento cumple una función y contiene todo lo necesario para que funcione una aplicación.



Pero si en algunos de esos departamentos se necesita agregar o modificar algo o necesita alguna actualización, la aplicación se rompe o sea los 4 departamentos dejan de funcionar.

Ciudad de las Aplicaciones



Ciudad de las Aplicaciones



Por eso se recomienda dividir las tareas en donde cada departamento tenga su propio edificio, así si algún departamento se necesita agregar o modificar algo o necesita alguna actualización, los demás edificios siguen funcionando.

Ventajas de microservicio

Usabilidad independiente: Cada microservicio puede ser escalado de manera independiente, lo que permite ajustar los recursos según la demanda específica de cada servicio.

Despliegue independiente: Los microservicios pueden ser desplegados de manera independiente, lo que facilita la entrada continua y la actualización de la aplicación sin afectar a otros servicios.

Facilidad de mantenimiento: Los microservicios permiten a los equipos de desarrollo trabajar de manera más independiente en cada servicio, lo que facilita la gestión del código y la corrección de errores.

Los microservicios permiten a los equipos de desarrollo trabajar de manera más independiente en cada servicio, lo que facilita la gestión del código y la corrección de errores.

Flexibilidad tecnológica: Cada microservicio puede ser desarrollado utilizando diferentes tecnologías y lenguajes de programación según lo que mejor se adapte a su función específica.

Tolerancia a fallos: la arquitectura de microservicios facilita la tolerancia a fallos, ya que un fallo en un servicio no afecta necesariamente a otros servicios.

Desventajas de microservicio

La gestión de múltiples microservicios puede ser compleja, ya que requiere herramientas y procesos adicionales para la monitorización, el descubrimiento, la gestión de la configuración y la coordinación entre servicios. Mayor complejidad de testing: la prueba de aplicaciones basadas en microservicios puede ser más compleja, ya que se deben probar no solo los servicios individualmente, sino también las interacciones entre ellos.

Arquitectura basada en capas

La arquitectura basada en capas es un enfoque comúnmente utilizado para diseñar sistemas de software, además se puede aplicar igualmente en el diseño y desarrollo de un microservicio.

La idea de implementar la arquitectura basada en capas, es segmentar responsabilidades en distintas capas.

En esta arquitectura tenemos 3 capas básicas:

Capa de presentación: En un microservicio, la capa de presentación suele ser mínima o incluso inexistente, ya que los microservicios generalmente no tienen un interfaz de usuario directa.

Sin embargo, en algunos casos un microservicio podría tener una API de interfaz de usuario para interactuar con otras partes del sistema o con aplicaciones externas.

En este caso, esta capa podría consistir en los controladores que exponen esta API.

Capa de logica de negocio: En esta capa,es donde se implementa la lógica de negocio específica de la aplicación.

En el contexto de un microservicio,esta capa se compone principalmente de servicios,que son las clases que implementan la lógica de negocio específica del servicio y proporcionan una interfaz para interactuar con ella.

Esto significa que los servicios son responsables de implementar las reglas de negocio,realizar cálculos,validar datos y coordinar las operaciones necesarias para cumplir con los requisitos del negocio.

Capa de acceso a datos: Esta capa es responsable de interactuar con la base de datos u otros sistemas de almacenamiento de datos.

En un microservicio,cada servicio podría tener su propia base de datos o compartir una base de datos con otros microservicios,dependiendo de la necesidad y de la arquitectura específica.

Capas adicionales

Capa de integración: Esta capa se utiliza para integrar el microservicio con otros sistemas externos o con otros microservicios dentro de la misma arquitectura de microservicios.

Puede incluir componentes de comunicación como clientes HTTP,clientes de servicios web,colas de mensajes,etc.

Capa de seguridad: Esta capa se encarga de implementar medidas de seguridad,como la autenticación y la autorización,para proteger el microservicio contra accesos no autorizados.

Puede incluir componentes como filtros de seguridad,tokens de acceso,etc.



Capa de presentación en donde se encuentra el controllers. En la capa de acceso a datos está el paquete main,exception y servis. Capa de acceso a datos está la base

de datos. En la capa de integración se encuentra el paquete configuración. Capa de seguridad se encuentra paquete configuración.

¿Que es la capa de servicios?

La capa de servicios, también conocida como Business Rule o Engine, es crucial en una arquitectura basada en capas. Esta capa es donde reside la lógica del negocio, es decir, la funcionalidad principal de nuestro programa.

Por ejemplo, en una aplicación de productos, la capa de servicios gestionará la lógica de los productos, mientras que la capa de presentación mostrará los datos al usuario.

API/API REST: Una API (interfaz de programación de aplicaciones), es un conjunto de definiciones y protocolos que facilitan la comunicación y la interacción entre diferentes componentes de software.

Las cuales actúan como puentes que permiten que aplicaciones y sistemas se comuniquen de manera eficiente y coherente.

La palabra aplicación se refiere a cualquier software con una función distinta. La interfaz puede considerarse como un contrato de servicio entre 2 aplicaciones. Este contrato define cómo se comunican entre sí mediante solicitudes y respuestas.

Arquitectura API: La arquitectura de una API se refiere a las reglas que rigen cómo se comparten los datos con los clientes.

Las API tienen distintos tipos de arquitectura, las cuales suelen explicarse en términos de cliente servidor. Donde la aplicación que envía la solicitud se llama cliente, y la que envía la respuesta se llama servidor.

- Algunas de las arquitecturas más comunes de una API son: SOAP
- RPC
- WebSocket
- REST

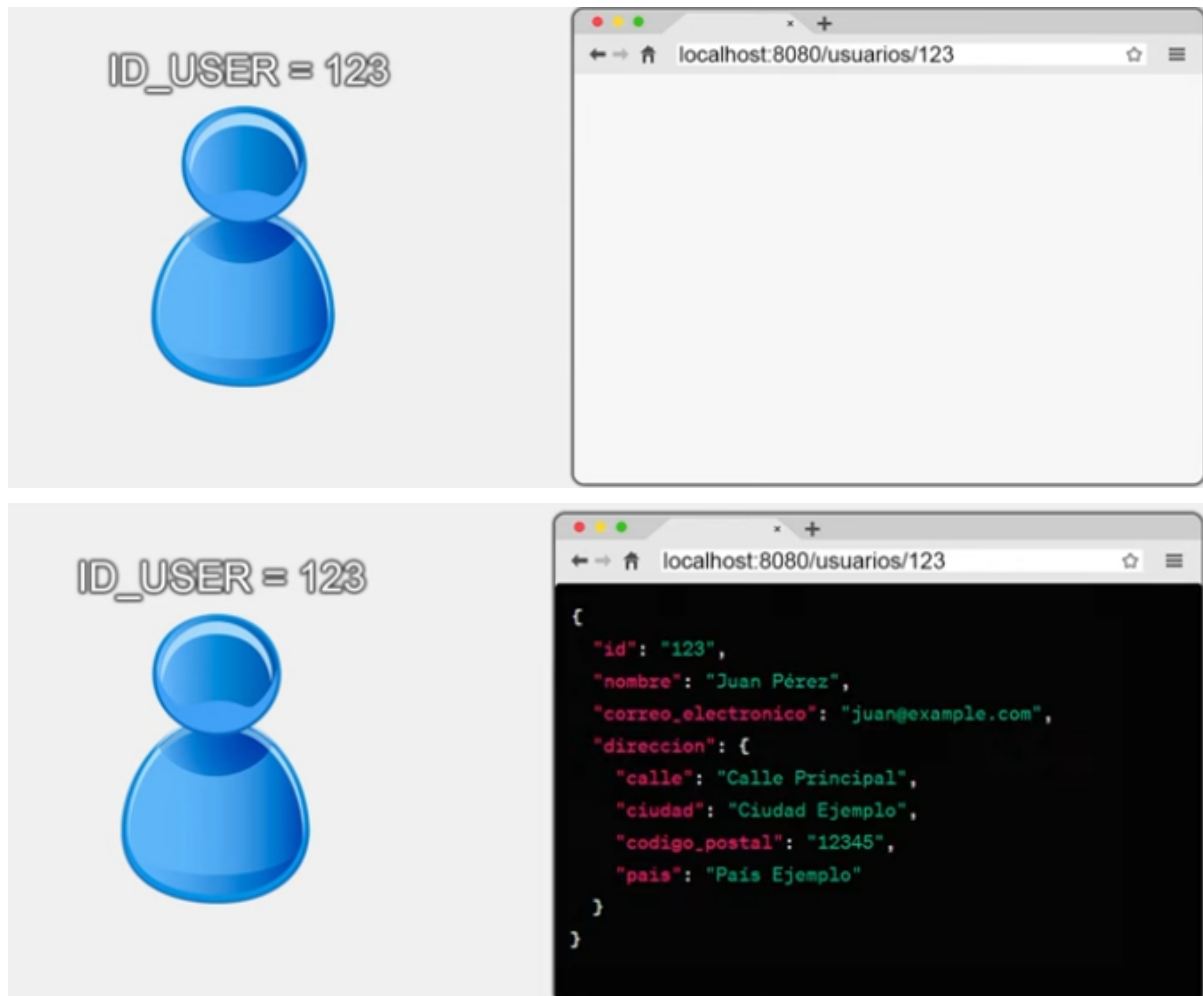
REST: REST o transferencia de estado representacional, es la arquitectura más común para APIs debido a su escalabilidad y facilidad de implementación. REST también puede denominarse RESTful.}

La arquitectura REST está basada en estándares web que utilizan métodos HTTP (GET, POST, PUT, PATCH, DELETE) para realizar operaciones CRUD (Crear, Leer, Actualizar, Borrar) en recursos que son identificados por URLs. REST utiliza el concepto de recursos y manipulación de estos recursos a través de interfaces uniformes.

Donde los recursos son entidades de información que son identificadas mediante URLs.

Estos recursos pueden ser cualquier cosa que pueda ser denominada y a la que se puede acceder a través de la red, como datos, archivos, servicios, etc.

Cada recurso en un arquitectura REST tiene una representación asociada, que puede ser en diferentes formatos como JSON, XML, HTML, entre otros. Esta representación es la información que se envía o se recibe cuando se accede al recurso a través de la API REST.



Con la URL se podría obtener los datos del usuario en formato JSON. De esta manera hemos aprendido, que los recursos en una arquitectura REST son entidades de información identificadas por URLs y pueden ser accedidos y manipulados a través de los métodos HTTP para realizar operaciones CRUD.

Restricciones de una API REST

Las API REST siguen restricciones clave que indican las reglas y condiciones que deben seguirse para cumplir con el estilo arquitectónico REST.

Separación cliente-servidor: esto permite que los cambios en cualquiera de los extremos no repercutan en el opuesto, favoreciendo la escalabilidad y la flexibilidad.

Interfaz uniforme: todas las peticiones y respuestas pertenecen a un identificador uniforme de recursos(URI),lo que significa que todas tienen el mismo formato y se transmiten a través de protocolos HTTP.

Sin estado: las aplicaciones del servidor no pueden almacenar datos de las peticiones de los clientes y cada petición es independiente de la siguiente.

Sistema en capas: dado que el servidor y los clientes están desacoplados,las solicitudes y respuestas se envían a través de múltiples aplicaciones,lo que requiere que tengan un formato coherente.Además,la API REST está diseñada para que ni el cliente ni el servidor puedan identificar si se está comunicando con una aplicación final o con un intermediario.

Almacenamiento en caché: el objetivo es ayudar a reducir las restricciones de la falta de estado,mejorar el rendimiento del lado del cliente,aumentar los resultados del lado del servidor y, en general,reducir el tiempo medio de respuesta.
En conclusión,una API es un conjunto de reglas y definiciones que permite que diferentes aplicaciones se comuniquen entre sí.
Mientras que una API REST es un estilo arquitectónico para diseñar redes y servicios web que utiliza el protocolo HTTP para acceder y manipular recursos.

Enrutamiento

El enrutamiento,también conocido como asignación de URL,es el proceso mediante el cual Spring Boot decide que método de controlador ejecutar en respuesta a una solicitud HTTP específica.

Esto se logra mediante la asignación de URLs a métodos de controlador específicos con base a las anotaciones utilizadas en esos métodos.

En Spring Boot,se utilizan varias anotaciones de mapeo para definir que método de controlador maneja una solicitud HTTP específica.

Coincidencia de patrones URL: Spring Boot utiliza enfoque basado en patrones para asignar solicitudes a métodos de controlador.

Esto significa que puede tener múltiples métodos de controlador con la misma ruta de acceso base,pero diferentes patrones adicionales,como parámetros de ruta o extensiones para diferenciarlos.

Clase ResponseEntity

La clase ResponseEntity se utiliza para representar toda la respuesta HTTP,incluyendo el cuerpo,los encabezados y el estado.

Lo cual es conveniente,para controlar de manera detallada cómo se construyen y devuelvan las respuestas HTTP desde los controladores de Spring.

Gracias a la clase ResponseEntity,podemos retornar tanto el contenido generado

por este servicio, así como proporcionar información adicional como el código de estado HTTP.

La clase `ResponseEntity`, es considerada una clase "Wrapper" (envoltorio), ya que encapsula un objeto dentro de sí misma

Ejemplo:

```
@GetMapping
public ResponseEntity<List<Customer>> getCustomers(){
    return ResponseEntity.ok(customers);
}
```

Código de respuesta HTTP

Los códigos de respuesta HTTP son códigos numéricos estándar que los servidores web envían a los clientes (navegadores, aplicaciones, etc.) para indicar el resultado de una solicitud HTTP.

Estos códigos ayudan a entender si una solicitud fue exitosa, si hubo errores, o si se necesita alguna acción adicional.

Código HTTP	Descripción	Uso típico
200	OK	Respuesta exitosa
201	Created	Recurso creado exitosamente
204	No Content	Respuesta exitosa sin contenido
400	Bad Request	Solicitud no válida
401	Unauthorized	Falta de autenticación o credenciales inválidas
403	Forbidden	Acceso denegado
404	Not Found	Recurso no encontrado
405	Method Not Allowed	Método no permitido para el recurso
409	Conflict	Conflicto en la solicitud
422	Unprocessable Entity	Entidad no procesable
500	Internal Server Error	Error interno del servidor

Cumplimiento de las convenciones REST

En el diseño de APIs RESTful, es una buena práctica atender una solicitud POST mostrando la URI del recurso recién creado.

Este enfoque sigue las convenciones REST y proporciona varios beneficios importantes.

Cuando un cliente realiza una solicitud POST para crear un nuevo recurso, la respuesta debe incluir la URI del recurso creado.

Esto se logra utilizando el código estado HTTP 201 Created junto con el encabezado Location, que indica la ubicación del nuevo recurso.

URI: Una URI (Identificador de Recursos Uniformes) es una cadena de texto que identifica de manera única un recurso en internet.

Piensa en una URI como una dirección que te dice donde encontrar algo. Este "algo" puede ser un documento, una imagen, un video, o cualquier otro recurso disponible en la web.

URL: Una URL (Localizador de Recursos Uniformes) es un tipo específico de URI. Una URL no solo identifica un recurso, sino que también proporciona una forma de localizarlo en Internet.

Una URL es la dirección completa que ves en la barra de direcciones de tu navegador web cuando visitas un sitio web.

Diferencia entre URI y URL

Identificación Y Localización

URI: Identifica un recurso de manera única.

URL: Identifica y proporciona la forma de localizar el recurso.

Ejemplo de URI:

urn:isbn:0451450523 este es un identificador para un libro específico, pero no te dice donde encontrarlo.

Ejemplo de URL: https://www.example.com/index.html este es un identificador y también te dice como llegar al recurso a través de un navegador web.

En resumen, todas las URL son URI, pero no todas las URI son URL. La URL es una forma más específica de URI que además de identificar un recurso te dice exactamente cómo acceder a él en internet.

Versionado y estructuración de URLs

El versionado y la estructuración de URLs en APIs RESTful son prácticas esenciales para asegurar que tu API sea mantenible, escalable y fácil de consumir.

Estas prácticas ayudan a manejar cambios en la API sin interrumpir a los clientes existentes y aseguran una comunicación clara sobre las diferentes versiones y funcionalidades de la API.

Convención de URL

La convención de URL es una forma estandarizada de estructurar los endpoint de una API para que sean intuitivos y fáciles de usar.

Esto incluye el uso de rutas descriptivas, verbos HTTP y parámetros claros.

Versionado de API

El versionado de API es una práctica que permite que diferentes versiones de una API coexistan simultáneamente.

Esto es crucial para asegurar que los cambios en la API no rompan las aplicaciones que dependen de versiones anteriores.

Tipos de versionados

1. Versionado en la URL: Incluye la versión en la ruta de la URL. El versionado en la URL es el método más común y fácil de implementar. Consiste en incluir la versión de la API directamente en la ruta de URL.
 2. Versionado en el encabezado: Utiliza encabezados HTTP para identificar la versión.
 3. Versionado en parámetros de consulta: Utiliza parámetros de consulta en la URL. **Ejemplo:** <http://localhost:8080/sistema/API>
 4. API Path: Es una ruta adicional que indica que los recursos siguientes pertenecen a la API de la aplicación. API es una convención común para diferenciar los endpoints de la API de otros tipos de recursos que la aplicación podría servir.
Ejemplo: Esto ayuda a distinguir entre endpoints que podrían ser utilizados por clientes web y móviles versus aquellos que podrían servir contenido estático o páginas HTML.
 5. Versión de la API: v1 indica la versión de la API que se está utilizando. Este número permite a los desarrolladores versionar la API y realizar cambios en futuras versiones sin afectar a los clientes que dependen de la versión actual.
Ejemplo: En el futuro podrías tener /v2,/v3,etc,para indicar nuevas versiones de la API con posibles cambios,mejoras o nuevos endpoints.
Ejemplo: <http://localhost:8080/sistema/api/v1>
- Obtener todos los clientes: GET/api/v1/clientes
 - Obtener un cliente por nombre_usuario:
GET/api/v1/clientes/{nombre_usuario}

- Crear un nuevo cliente: POST/api/v1/clientes
- Actualizar un cliente: PUT/api/v1/clientes

Programación orientada a Interfaces

La programación orientada a interfaces es un enfoque dentro de la programación orientada a objetos(POO) que se centra en el uso de interfaces para definir el comportamiento que las clases deben implementar.

La programación orientada a interfaces permite definir comportamientos mediante contratos, promoviendo la separación entre la definición y la implementación, mejorando la flexibilidad y la escalabilidad del sistema.

Es una técnica clave para crear sistemas de software más robustos y adaptables.

¿Qué es una interfaz?

Una interfaz es un contrato que especifica un conjunto de métodos que una o más clases deben proporcionar, sin definir cómo se deben implementar estos métodos.

Beneficios de usar interfaces:

Abstracción: Permite separar la definición del comportamiento(en la interfaz) de la implementación concreta(en las clases).

Esto facilita la reutilización del código y reduce el acoplamiento.

Flexibilidad: permite cambiar las implementaciones sin afectar el código que utilizan las interfaces.

Puedes intercambiar implementaciones diferentes sin modificar el código cliente.

Escalabilidad: facilita la adición de nuevas funcionalidades. Puedes agregar nuevas clases que implemente la misma interfaz sin alterar el código existente.

Inyección de dependencias

La inyección de dependencias es un patrón de diseño que permite a los objetos recibir sus dependencias desde el exterior en lugar de crearlas por sí mismos.

El objetivo de la inyección de dependencias, es promover un bajo acoplamiento entre componentes y facilitar el mantenimiento y la escalabilidad del código.

Para lograr la inyección de dependencias, es necesario utilizar una técnica basada en el principio de inversión de control(IOC).

La inversión de control(IOC) es el principio en el que se basa la inyección de dependencias, donde el control de la creación y gestión de objetos se invierte desde el código a un contenedor o framework(como Spring).

En el contexto de Spring, el contenedor de IOC es el mismo Spring.

Ya que un "contenedor" es un componente que se encarga de gestionar el ciclo de vida de los objetos, también conocidos como beans, incluida su creación, configuración y la inyección de dependencias. Spring incluye un contenedor IoC que gestiona los beans de la aplicación. Este contenedor es responsable de:

- Crear instancias
- Configurar dependencias
- Gestionar ciclos de vida de los beans

Cuando usamos inyección de dependencias le indicas al contenedor (en este caso, Spring) que se encargue de crear e inyectar las instancias de las dependencias necesarias en tus clases, en lugar de hacerlo tú manualmente. Inversión de control: al delegar esta responsabilidad al contenedor IoC de Spring, estás invirtiendo el control que típicamente tendría tu aplicación sobre la creación y gestión de objetos.

Esto permite que Spring gestione automáticamente las instancias de las dependencias y las inyecta en tus clases.

Cuando utilizamos inyección de dependencia en Spring, estamos aplicando el principio de inversión de control (IoC).

Lo que significa que el contenedor de Spring se encarga de generar automáticamente las instancias de las dependencias y las inyecta en nuestras clases.

Tipo de inyección de dependencias:

Existen 3 tipos de inyección de dependencias:

- Inyección por Constructor
- Inyección por Setter
- **Inyección por Campo:** Cuando utilizamos inyección de dependencias por campo, las dependencias se inyectan directamente en los campos de la clase, es decir, en sus atributos.